

Stochastic NeuralGCM Ensemble Generation Week 1

Project Goal

Our research project studies seasonal prediction of Western U.S. snow water equivalent (SWE). One part of the project requires a stochastic weather/climate emulator that can start from **one given atmospheric initial condition** and generate multiple plausible future atmospheric trajectories.

For this task, you will work with **Stochastic NeuralGCM**. Here, “stochastic” means that we hold the initial atmospheric state fixed, change the model's random seed, and generate different plausible future realizations:

$$X_0 \xrightarrow{\epsilon_1} X_{1:T}^{(1)}, \quad X_0 \xrightarrow{\epsilon_2} X_{1:T}^{(2)}, \quad \dots, \quad X_0 \xrightarrow{\epsilon_N} X_{1:T}^{(N)}.$$

The eventual goal is to generate an ensemble of possible atmospheric and precipitation evolutions from the same historical initial condition.

Task 0 — Obtain Perlmutter/NERSC Access

Create a NERSC account and obtain access to **Perlmutter**.

Official instructions:

<https://docs.nersc.gov/accounts/>

Once your account is activated, learn how to SSH into Perlmutter and perform one successful SSH login test.

Task 1 — Create the Project GitHub Repository

Create a **private GitHub repository** for this project and invite:

hongyuchen1030

as a collaborator.

The default **README.md** is sufficient for now.

After creating the repository, configure GitHub SSH access from your Perlmutter account and successfully clone the private repository onto Perlmutter using Git/SSH.

Task 2 — Understand NeuralGCM and Stochastic Forecasting

The following is the useful information you can feed in your LLMs models as prompts.

Read the NeuralGCM paper and documentation and understand:

- What NeuralGCM is.
- What “stochastic” means in NeuralGCM.
- How the stochastic model can generate different future trajectories from the same initial atmospheric state.
- How NeuralGCM is initialized and rolled forward.
- What information must be supplied to the model during a forecast.

Start with:

- [NeuralGCM paper](#)
- [NeuralGCM documentation](#)
- [Forecasting quick start](#)
- [Pre-trained model checkpoints](#)

The conceptual goal is to understand how we eventually perform:

$$X_0 \xrightarrow{\text{seed}_1} \text{Future}_1, \quad X_0 \xrightarrow{\text{seed}_2} \text{Future}_2,$$

while keeping (X_0) unchanged.

Task 3 — Set Up the NeuralGCM Model

For this project, use the **2.8° stochastic NeuralGCM model trained to predict precipitation**:

```
gs://neuralgcm/models/v1_precip/stochastic_precip_2_8_deg.pkl
```

Official checkpoint documentation:

<https://neuralgcm.readthedocs.io/en/latest/checkpoints.html>

Follow the official NeuralGCM forecasting example to understand how to load the checkpoint and construct the corresponding `PressureLevelModel`. Then set up the model inside your project repository on **Perlmutter** and perform a small smoke test to confirm that the checkpoint can be loaded and executed successfully.

Before running the model, familiarize yourself with how jobs should be executed on Perlmutter. In particular, understand the difference between **interactive jobs** and **Slurm batch jobs**, how compute resources should be requested, and the recommended practices for running GPU/compute workloads. Use the following NERSC documentation as reference:

- Job overview: <https://docs.nersc.gov/jobs/>
- Job best practices: <https://docs.nersc.gov/jobs/best-practices/>
- Interactive jobs: <https://docs.nersc.gov/jobs/interactive/>

For the first smoke test, determine whether an interactive allocation or a submitted Slurm job is more appropriate, request the necessary resources, and document the command or job script you used.

If you plan to use a **coding agent** directly on Perlmutter to help with development or debugging, read the NERSC coding-agent documentation first and follow its requirements and recommended practices:

<https://docs.nersc.gov/development/coding-agents/>

The goal of this task is to confirm that you can **load the NeuralGCM checkpoint, understand how NeuralGCM jobs should be executed on Perlmutter, and successfully complete one minimal test run using the appropriate NERSC workflow**.

Expected Deliverable

At the end of the week, write a short summary in your GitHub repository as a Markdown document describing what you learned during the setup process.

Document your own understanding of NeuralGCM, including:

- What information must be supplied to the model before and during a forecast.
- What input variables and forcing variables are required.
- What output variables/information the model generates.
- How the stochastic NeuralGCM generates different future realizations from the same initial atmospheric condition.
- What you learned while setting up and running the model.
- Any remaining questions, uncertainties, or issues you encountered.

Preferably, include a simple flowchart showing the workflow, for example:

Initial atmospheric state

- required forcing variables ↓ Stochastic NeuralGCM
- different random seeds ↓ Multiple possible future atmospheric trajectories ↓ Atmospheric variables + precipitation output

You can expand the flowchart to show the actual required variables once you have inspected the checkpoint.

If a flowchart becomes too complicated, it is completely fine to describe the workflow clearly in your own words instead. The main goal is to demonstrate that you understand what NeuralGCM needs as input, what it produces as output, how its stochastic generation works, and what you learned or still have questions about after completing the setup.

NeuralGCM Ensemble Generation — Week 2

As described in the NeuralGCM documentation, NeuralGCM has two types of models:

1. **Non-stochastic model:** each input entry generates one output realization.
2. **Stochastic model:** one input entry can generate multiple output realizations.

This week, we will try to use NeuralGCM to synthesize the weather data needed for snow-data generation.

Do not get intimidated by the data dump below. This week, we only aim to achieve ONE running pipeline test. This means you only need to pick **one model** among the four models below and **one water year**.

For the one-water-year proof test, we can start with:

- **Model:** MIROC6
- **Member:** r1i1p1f1
- **Experiment:** ssp370
- **Water year:** WY2016

WY2016 means that the simulation starts in fall 2015 and runs through April 1, 2016.

We previously selected MIROC6 because it provides a clean, available, internally matched CMIP6–WUS-D3 pair for this proof-of-concept. We already have the corresponding CMIP6 predictor data and matching WUS-D3 d02 SWE simulation, so we can test the full pipeline without first solving a broader data-availability problem.

However, **feel free to pick a different model and a different year**. The goal this week is simply to demonstrate one complete working example of the lagged-ensemble data-inflation pipeline.

The Data

We use WUS-D3 d02-generated SWE as our target. This SWE is the result of a physical weather simulation.

Relationship between CMIP6 and WUS-D3 d02

Think of **CMIP6** → **WUS-D3** as a two-stage physical climate simulation pipeline.

We use **four CMIP6 global climate models**. Each model produces its own simulation of Earth's climate on a global grid. From each CMIP6 simulation, we extract the 19 atmospheric, ocean, and land variables that we are interested in:

```
tos, siconc, rlut, zg_500, ta_850,
ua_850, va_850, ua_200, va_200,
hus_850, psl, tas,
zg_50, ta_50, ua_50, va_50,
mrso, thetao_50m, thetao_100m.
```

These variables are **global**: they describe climate conditions over the Earth rather than only over the Sierra Nevada.

Because we use four different CMIP6 models, for the same simulated period we effectively have:

CMIP model 1 → X_1^{19} , CMIP model 2 → X_2^{19} , CMIP model 3 → X_3^{19} , CMIP model 4 → X_4^{19} .

The four models therefore provide **four different physically generated realizations of the same 19 climate variables**.

Each of these CMIP6 simulations has a corresponding **WUS-D3 regional simulation**. WUS-D3 dynamically downscales the CMIP6 climate simulation over the western United States at much higher spatial resolution. For our project, we use the **WUS-D3 d02** domain.

The relationship can be visualized as:

```
CMIP6 global climate model #1 → WUS-D3 d02 #1 → Sierra SWE #1
CMIP6 global climate model #2 → WUS-D3 d02 #2 → Sierra SWE #2
CMIP6 global climate model #3 → WUS-D3 d02 #3 → Sierra SWE #3
CMIP6 global climate model #4 → WUS-D3 d02 #4 → Sierra SWE #4
```

GLOBAL CLIMATE
19 variables

REGIONAL DOWNSCALING
high-resolution SWE

For our machine-learning dataset, the pairing is therefore essentially:

<u>19 global CMIP6 variables</u>	→	<u>corresponding WUS-D3 d02 Sierra SWE</u>
ML input		ML target

The important point is that **CMIP6 and WUS-D3 are not two unrelated datasets**. Each WUS-D3 simulation is associated with a particular parent CMIP6 simulation. Therefore, when constructing a training sample, we must

preserve this parent relationship: CMIP model 1's 19-variable climate state is paired with the SWE produced by its corresponding WUS-D3 simulation, model 2 with model 2, and so on.

Data Locations and CMIP6–WUS-D3 Pairings

CMIP6 19-Variable Global Inputs

The regridded 1.5° CMIP6 predictor fields are located at:

```
/pscratch/sd/h/hyvchen/Snow-Predication-at-Sierra/cmip6_regridded_1p5deg/raw/
```

WUS-D3 d02 Daily SWE

The corresponding **daily WUS-D3 d02 SWE (snow)** files are located at:

```
/global/cfs/projectdirs/m3522/datalake/WUS-D3/daily/<dataset_id>_{historical,ssp370}_bc/postprocess/d02/snow.daily.*.nc
```

Here, **<dataset_id>** identifies the WUS-D3 simulation associated with a particular CMIP6 parent model and ensemble member.

For example, for **MIROC6 (r1i1p1f1)**, the dataset ID is:

```
miroc6_r1i1p1f1
```

Therefore, the corresponding WUS-D3 directories follow the pattern:

```
/global/cfs/projectdirs/m3522/datalake/WUS-D3/daily/miroc6_r1i1p1f1_{historical,ssp370}_bc/postprocess/d02/
```

The daily SWE files inside these directories follow:

```
snow.daily.*.nc
```

For example, a previously verified MIROC6 WUS-D3 d02 daily SWE file is:

```
snow.daily.miroc6_r1i1p1f1.ssp370.bias-correct.d02.2015.nc
```

These are the **daily SWE fields**, not the processed April 1 SWE targets used by the CNN.

Parent Model Pairings

Each CMIP6 global simulation is paired with its corresponding WUS-D3 regional downscaling:

CMIP6 Parent Model	CMIP6 Member	Corresponding WUS-D3 Dataset
EC-Earth3	r102i1p1f1	ec-earth3_r1i1p1f1_2_{historical,ssp370}*bc
MIROC6	r1i1p1f1	miroc6_r1i1p1f1*{historical,ssp370}*bc
MPI-ESM1-2-HR	r3i1p1f1	mpi-esm1-2-hr_r3i1p1f1*{historical,ssp370}*bc
TaiESM1	r1i1p1f1	taiesm1_r1i1p1f1*{historical,ssp370}_bc

The pairing must be preserved when constructing the training data: the global predictor fields from each CMIP6 parent simulation must be matched with the SWE generated by its corresponding WUS-D3 d02 regional simulation.

Conceptually:

```
CMIP6 EC-Earth3      → WUS-D3 EC-Earth3      → Sierra SWE
CMIP6 MIROC6         → WUS-D3 MIROC6         → Sierra SWE
CMIP6 MPI-ESM1-2-HR  → WUS-D3 MPI-ESM1-2-HR  → Sierra SWE
CMIP6 TaiESM1        → WUS-D3 TaiESM1        → Sierra SWE
```

Thus, the CMIP6 global climate fields and WUS-D3 regional SWE are physically paired simulations rather than independent datasets.

Non-Stochastic NeuralGCM: Lagged Ensemble

We will eventually use an idea called a **lagged ensemble**. You can learn more about this approach in the paper linked in the task description.

The basic idea is that instead of representing November using only one fixed monthly window, we can construct multiple slightly shifted windows. For example:

```
Nov 1-7 average → November ensemble member #1
Nov 2-8 average → November ensemble member #2
Nov 3-9 average → November ensemble member #3
...
```

Each shifted window represents a slightly different initial condition. When passed through the **non-stochastic NeuralGCM**, each input produces one corresponding weather realization. Collectively, these realizations would form the lagged ensemble.

Important: Only Generate ONE Ensemble Member This Week

You do NOT need to construct the full sliding window or generate the full November ensemble for this task.

For this proof-of-concept, choose only:

- **ONE CMIP6/WUS-D3 parent model**

- **ONE water year**
- **ONE sliding window**
- **ONE NeuralGCM ensemble member**

For example, you may use:

```
Model:      MIROC6
Member:     r1i1p1f1
Experiment:  ssp370
Water year: WY2016
Window:     Nov 1-7, 2015
```

Use that **single sliding-window input** to run the non-stochastic NeuralGCM once.

Therefore, the expected workflow for this week's proof is simply:

```
ONE model
↓
ONE water year
↓
ONE November sliding window
↓
ONE non-stochastic NeuralGCM run
↓
ONE generated weather realization
↓
ONE precipitation forcing
+
ONE 2-m temperature forcing
↓
Snow-17
↓
ONE corresponding SWE realization
```

In other words, **you are implementing one member of what will eventually become the full lagged ensemble.**

Required SWE Output Grid

The final Snow-17 SWE field should be produced on the **WUS-D3 d02 spatial grid**, rather than retaining the native resolution of the weather-generation model.

Target grid:

```
WUS-D3 d02
Native WRF Lambert grid
Approximately 9 km horizontal resolution
Grid size: 340 × 270
```

NeuralGCM and other weather-generation models may have different native horizontal resolutions. Therefore, appropriately map/regrid the required meteorological forcing to the WUS-D3 d02 grid before or as part of the Snow-17 calculation.

The desired output of this proof is:

```
ONE NeuralGCM weather realization
  ↓
precipitation + 2-m temperature
  ↓
mapped to WUS-D3 d02 spatial grid
  ↓
Snow-17
  ↓
ONE SWE realization on the 340 × 270 WUS-D3 d02 grid
```

Do **not** simply return SWE at the native NeuralGCM resolution.

For later comparison with the existing CNN target, April 1 SWE can additionally be area-averaged over:

35°N–42°N, 122.5°W–118°W

but preserve the full WUS-D3 d02 SWE grid as the primary output of this experiment.